2-Marks Questions (All Units)

1. State any four salient features of the 8086 microprocessor.

1. It is a **16-bit microprocessor**.
2. It has **20 address lines** to access 1 MB memory.
3. It has **16 data lines**.
4. It supports **pipelining** using BIU and EU.

---

2. Define Procedure and write its syntax.

A **Procedure** is a group of instructions used to perform a specific task and can be called many times.

Syntax:

```
PROC_NAME PROC NEAR/FAR
  ; instructions
  RET
PROC_NAME ENDP
```

---

3. Define Macro and write its syntax.

A **Macro** is a set of instructions given a name, expanded wherever called.

Syntax:

```
NAME MACRO
  ; instructions
ENDM
```

---

4. State the use of STC and CMC instructions of 8086.

- **STC**: Sets Carry Flag = 1.
- **CMC**: Complements Carry Flag (0 to 1 or 1 to 0).

---

5. Define Stack Pointer (SP) and Base Pointer (BP) registers.

- **SP (Stack Pointer):** Stores offset address of top of stack.
- **BP (Base Pointer):** Used to access data from stack memory.

---

6. Compare MUL and IMUL instructions of 8086 (4 points).

MUL	IMUL
Used for unsigned multiplication	Used for signed multiplication
Multiplies positive numbers	Multiplies positive and negative numbers
Result is unsigned	Result is signed
No sign bit considered	Sign bit considered

---

7. State the function of the following pins: ALE, READY.

- **ALE (Address Latch Enable):** Used to latch lower byte address from multiplexed bus.
- **READY:** Used to insert wait states for slow devices.

---

8. State the function of an Assembler and a Linker.

- **Assembler:** Converts assembly language program into machine code (.OBJ file).
- **Linker:** Combines object files and creates executable file (.EXE).

---

9. Differentiate between NEAR and FAR procedure calls (4 points).

NEAR Call	FAR Call
Calls procedure in same code segment	Calls procedure in different code segment
Only IP is changed	CS and IP both are changed
Returns by popping IP	Returns by popping CS and IP
Faster execution	Slower than near call

---

10. Explain the DAA instruction with its use in BCD addition.

DAA means Decimal Adjust after Addition.
It adjusts result after adding packed BCD numbers.

Use: Used after ADD instruction in BCD addition.

Example:

```
MOV AL,25H
ADD AL,38H
DAA
```

Result = 63H

---

11. State the use of CF (Carry Flag) and AF (Auxiliary Flag) in 8086.

- **CF (Carry Flag):** Indicates carry or borrow in arithmetic operation.
- **AF (Auxiliary Flag):** Indicates carry from lower nibble to upper nibble (used in BCD).

---

12. What is the role of the XCHG instruction?

XCHG instruction exchanges data between two registers or register and memory.

Example:

```
XCHG AX,BX
```

---

13. Explain DAA and CMP instructions of 8086.

- **DAA:** Adjusts result of BCD addition into valid BCD number.
- **CMP:** Compares two operands by subtraction without storing result.

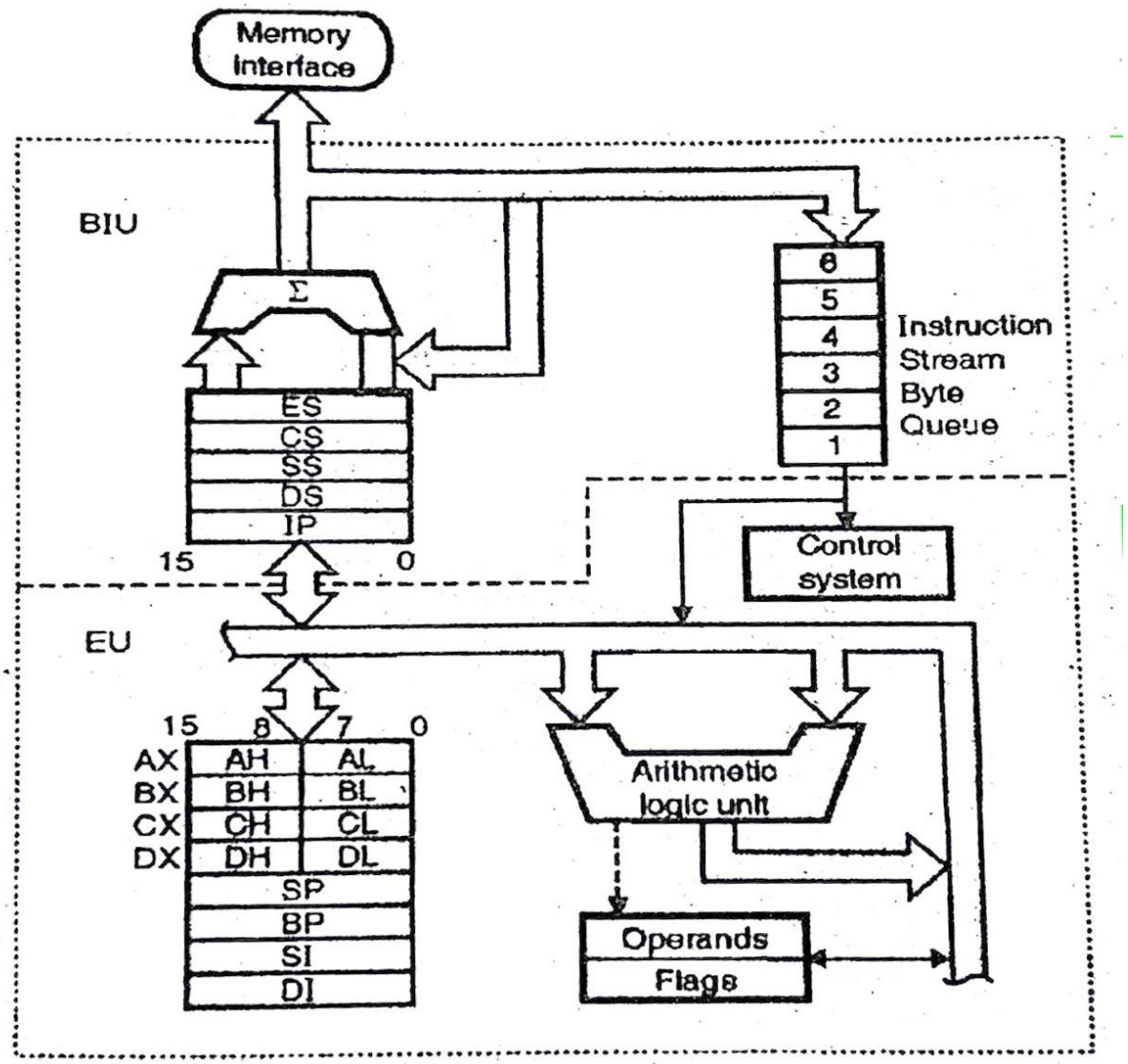
Example:

```
CMP AX,BX
```

4 & 6-Marks Questions (Unit-Wise)

Unit I: 8086 Microprocessor Architecture

1. Draw the functional block diagram of the 8086 microprocessor with all labels.

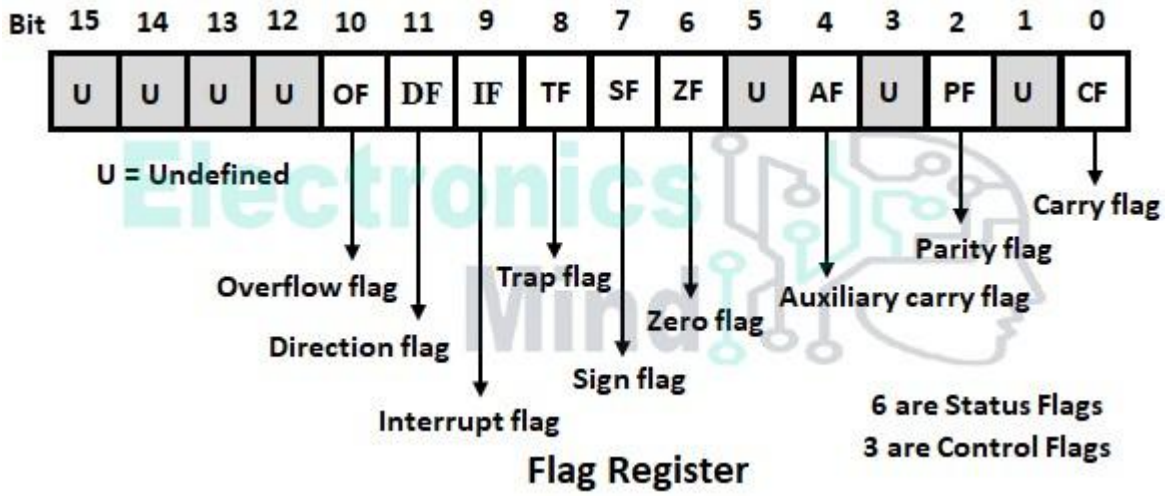


8086 internal architecture

Explanation: BIU fetches instructions and EU executes them.

---

2. Draw the flag register format of the 8086 microprocessor and explain any two flags.



Any Two Flags:

1. **CF (Carry Flag):** Set when carry occurs in addition or borrow in subtraction.
2. **ZF (Zero Flag):** Set when result becomes zero.

---

3. Explain the concept of memory segmentation in 8086 with neat diagram and advantages.

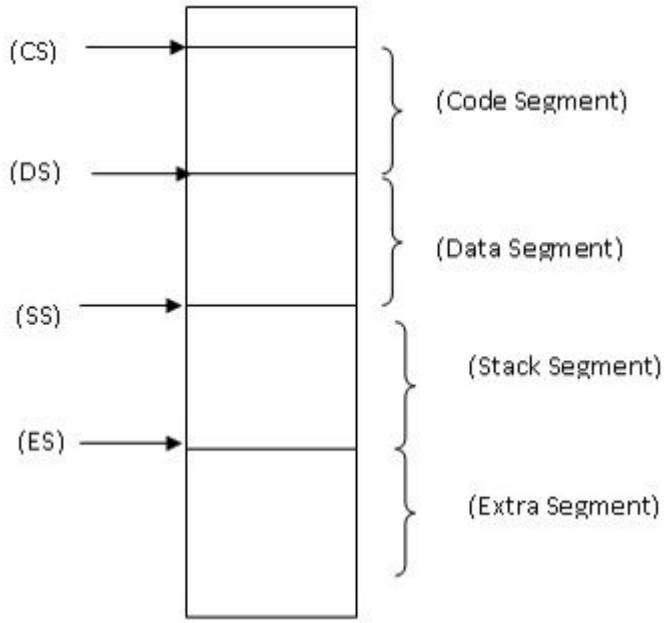
8086 divides 1 MB memory into segments of 64 KB each.

Types of segments:

- Code Segment (CS)
- Data Segment (DS)
- Stack Segment (SS)
- Extra Segment (ES)

```
   1 MB MEMORY
 +-----------------+
 | Code Segment    |
 +-----------------+
 | Data Segment    |
 +-----------------+
 | Stack Segment   |
 +-----------------+
 | Extra Segment   |
 +-----------------+
```

OR



Advantages:

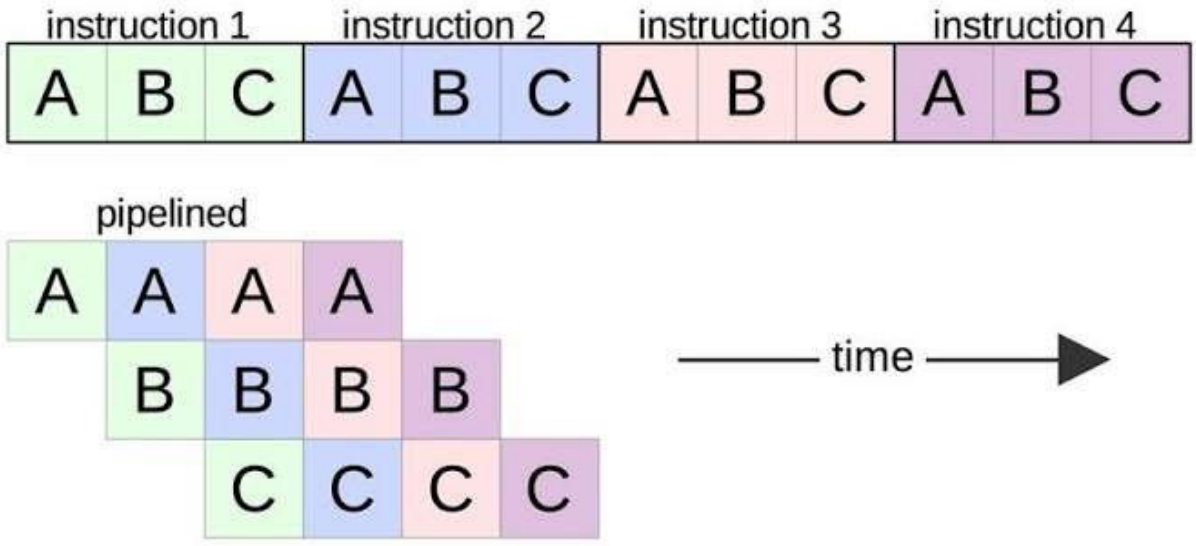
1. Efficient memory management.
2. Separate code and data storage.
3. Supports modular programming.
4. Easy relocation of programs.

---

4. Explain the concept of pipelining in 8086 with neat diagram. State advantages.

Pipelining means fetching next instruction while current instruction is executing.

Time --->



BIU stores instructions in 6-byte queue.

Advantages:

1. Increases speed.
2. Better CPU utilization.
3. Reduces waiting time.
4. Faster instruction execution.

---

5. Explain physical address generation in 8086. Define logical and effective address.

Physical address is generated by:

Physical Address = Segment Address × 10H + Offset Address

Example:
CS = 2000H, IP = 1234H

Physical Address = 2000H × 10H + 1234H = 21234H

Logical Address: Segment address + Offset address.
Example: 2000H:1234H

Effective Address: Offset address of memory location.

---

6. State function of pins: M/IO, MN/MX, BHE, HOLD.

- **M/IO:** Selects memory operation or I/O operation.
- **MN/MX:** Selects minimum mode or maximum mode.
- **BHE:** Enables higher byte of data bus.
- **HOLD:** Requests control of system bus from DMA device.

---

7. Differentiate between minimum and maximum mode of 8086 (9 points).

Minimum Mode	**Maximum Mode**
Used in single processor system	Used in multiprocessor system
Selected by MN/MX = 1	Selected by MN/MX = 0
Only one processor works	Multiple processors can work
Generates control signals internally	Uses external controller 8288
Simpler hardware design	Complex hardware design

Minimum Mode	Maximum Mode
Lower cost	Higher cost
No bus arbitration needed	Bus arbitration required
Suitable for small systems	Suitable for large systems
Example: Personal computer	Example: Networked system

Unit II: Programming Tools and Addressing Modes

1. Explain any six addressing modes of 8086 with suitable examples.

1. **Immediate Addressing Mode**
   Data is given directly in instruction.
   Example:

   MOV AL,25H

2. **Register Addressing Mode**
   Operand is in register.
   Example:

   MOV AX,BX

3. **Direct Addressing Mode**
   Memory address is directly given.
   Example:

   MOV AL,[1234H]

4. **Register Indirect Addressing Mode**
   Memory address stored in register.
   Example:

   MOV AL,[BX]

5. **Indexed Addressing Mode**
   Index register used for address.
   Example:

   MOV AL,[SI]

6. **Based Indexed Addressing Mode**
   Base and index registers used together.
   Example:

   MOV AL,[BX][SI]

---

2. Identify the addressing mode for following instructions.

1. MOV AL,46H → **Immediate Addressing Mode**
2. MOV BX,[1234H] → **Direct Addressing Mode**
3. ADD AL,[BX][SI] → **Based Indexed Addressing Mode**
4. MUL AL,BL → **Register Addressing Mode**

---

3. List and explain the program development steps in assembly language programming.

1. **Edit** – Write program using editor and save as .ASM file.
2. **Assemble** – Convert source code into object code using assembler.
3. **Link** – Link object file and create .EXE file.
4. **Load** – Load executable file into memory.
5. **Run** – Execute the program.
6. **Debug** – Find and remove errors.

---

4. Explain assembler directives with examples.

1. **DB** – Define Byte

   NUM DB 25H
2. **DW** – Define Word

   NUM DW 1234H
3. **EQU** – Assign constant value

   MAX EQU 10
4. **SEGMENT** – Starts segment

   DATA SEGMENT
5. **ENDS** – Ends segment

   DATA ENDS
6. **DUP** – Duplicate values

   ARR DB 5 DUP(0)
7. **ASSUME** – Associates segment register

   ASSUME CS:CODE, DS:DATA
8. **OFFSET** – Gives offset address

   MOV SI,OFFSET NUM

---

5. Describe the model of assembly language programming.

Assembly language program model contains:

1. **Data Segment** – Stores variables.
2. **Code Segment** – Stores instructions.
3. **Stack Segment** – Stores stack data.
4. **Procedure/Main Program** – Execution starts here.

Example:

```
.model small
.stack 100h
.data
msg db 'Hello$'
.code
main:
mov ax,@data
mov ds,ax
```

Output:

Hello

---

6. Explain assembly language program development tools.

1. **Editor** – Used to write and modify source program.
2. **Assembler** – Converts assembly language into machine code.
3. **Linker** – Combines object files and creates EXE file.
4. **Debugger** – Used to test and remove errors from program.

Unit III: 8086 Instruction Set

1. Describe any four bit-manipulation (shift/rotate) instructions of 8086 with suitable examples.

1. **SHL** – Shifts bits left by 1 position.

   SHL AL,1

2. **SHR** – Shifts bits right by 1 position.

   SHR AL,1

3. **ROL** – Rotates bits left.

   ROL AL,1

4. **ROR** – Rotates bits right.

   ROR AL,1

---

2. Explain any four string instructions of 8086 with suitable examples.

1. **MOVSB** – Moves byte from source to destination.

   MOVSB

2. **LODSB** – Loads byte from memory to AL.

   LODSB

3. **STOSB** – Stores AL into memory.

   STOSB

4. **CMPSB** – Compares two bytes of strings.

   CMPSB

---

3. Explain with examples any four logical instructions of 8086.

1. **AND** – Performs logical AND.

   AND AL,0FH

2. **OR** – Performs logical OR.

   OR AL,01H

3. **XOR** – Performs logical XOR.

   XOR AL,AL

4. **NOT** – Complements bits.

   NOT AL

---

4. Illustrate the use of any three branching instructions.

1. **JMP** – Unconditional jump.

   JMP NEXT

2. **JE** – Jump if equal.

   JE SAME

3. **JNZ** – Jump if not zero.

   JNZ LOOP1

---

5. Explain the XLAT and XCHG instructions with suitable examples.

1. **XLAT** – Translates byte using lookup table.

   XLAT

Uses BX as table base and AL as index.

2. **XCHG** – Exchanges data between operands.

   XCHG AX,BX

6. Differentiate between ROL and RCL instructions (9 points).

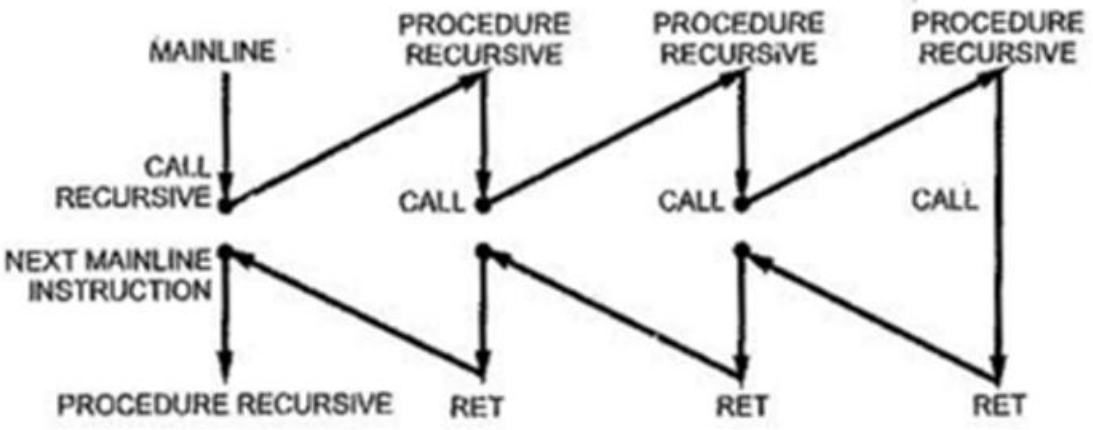
ROL	RCL
Rotate Left instruction	Rotate through Carry Left instruction
Bits rotate only within operand	Bits rotate through operand and Carry Flag
MSB moves to LSB	MSB moves to Carry Flag
Carry gets old MSB	Carry participates in rotation
Carry not used as input bit	Carry used as input bit
Rotation of 8/16 operand bits only	Rotation of 9/17 bits (with carry)
Used for simple circular rotation	Used for multi-byte rotation
Faster and simpler	Slightly complex
Example: ROL AL,1	Example: RCL AL,1

Unit V: Procedures and Macros

1. Explain Re-entrant and Recursive procedures with schematic diagram.

Re-entrant Procedure:

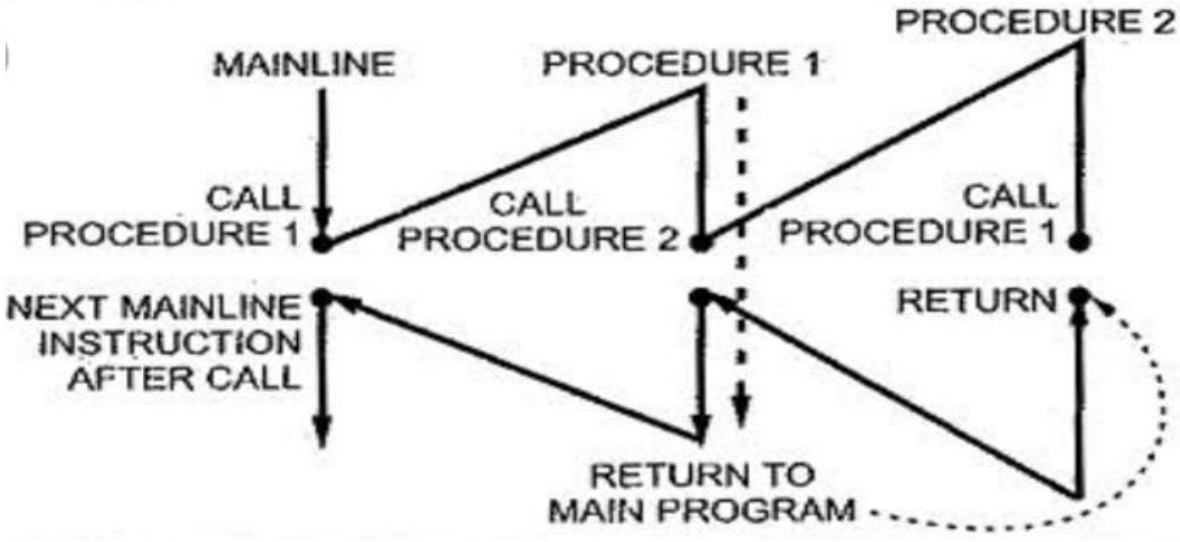
A **Re-entrant procedure** can be used by many programs/users at same time without changing its code.



Use: Shared memory programs.

Recursive Procedure:

A **Recursive procedure** calls itself again and again until condition is satisfied.



Use: Factorial, Fibonacci.

---

2. Compare Procedure and Macro with example and syntax (4 points).

Procedure	**Macro**
Called using CALL instruction	Called by name directly
Requires RET instruction	No RET needed
Saves memory	Uses more memory
Slower execution	Faster execution

Procedure Syntax:

```
ADD1 PROC
 ;code
 RET
ADD1 ENDP
```

Macro Syntax:

```
ADD1 MACRO
 ;code
ENDM
```

---

3. Differentiate between NEAR and FAR procedure calls (4 points).

NEAR Call	**FAR Call**
Calls within same segment	Calls different segment
Only IP changes	CS and IP change
Returns by popping IP	Returns by popping CS and IP
Faster	Slower

---

4. Describe with suitable example the concept of parameter passing in a procedure.

Parameter passing means sending data to procedure through registers, memory or stack.

Example using registers:

```
MOV AX,05H
MOV BX,03H
CALL ADDNUM

ADDNUM PROC
ADD AX,BX
RET
ADDNUM ENDP
```

Output:

AX = 0008H

Explanation: Values 5 and 3 passed through AX and BX registers, added in procedure.

Numericals & Assembly Language Programming (ALPs)

Part A: Calculations & Drafting Instructions

1. Physical Address Calculation: Calculate the 20-bit physical address. Given CS = 15AC H and IP = 8B21 H (or similar segment/offset values).

Given:
CS = 15ACH
IP = 8B21H

Formula:

Physical Address = Segment × 10H + Offset

= 15ACH × 10H + 8B21H
= 15AC0H + 8B21H
= 1E5E1H

Answer: 20-bit Physical Address = 1E5E1H

---

2. Instruction Drafting: Write appropriate single-line 8086 instructions to perform the following operations: [IMP] a.Multiply AL by 05H b.Rotate the contents of AX towards the left by 4 bits. c.Move 3000H in the BX register. d.Signed division of BL and AL. e.Add 100H to the contents of AX. f.Load SP register with FF00H.

a. Multiply AL by 05H

```
MOV BL,05H
MUL BL
```

b. Rotate contents of AX towards left by 4 bits

ROL AX,4

c. Move 3000H in BX register

MOV BX,3000H

d. Signed division of BL and AL

IDIV BL

e. Add 100H to contents of AX

ADD AX,100H

f. Load SP register with FF00H

MOV SP,FF00H

Part B: Highly Repeated ALPs (Assembly Language Programs)

1. Using a MACRO, write an assembly language program to solve equation $P = X^2 + Y^2$

```
.model small
.stack 100h
.data
X db 03h
Y db 04h
P dw ?

SQR MACRO NUM
MOV AL,NUM
MUL AL
ENDM

.code
main:
mov ax,@data
mov ds,ax

SQR X
MOV BX,AX

SQR Y
ADD AX,BX

MOV P,AX

mov ah,4ch
```

```
int 21h
end main
```

Output:

P = 0019H

Explanation: Calculates P = $3^2 + 4^2 = 25$.

---

2. Write an ALP for Z = (P+Q) * (R+S) using a MACRO

```
.model small
.stack 100h
.data
P db 02h
Q db 03h
R db 04h
S db 01h
Z dw ?

ADDN MACRO A,B
MOV AL,A
ADD AL,B
ENDM

.code
main:
mov ax,@data
mov ds,ax

ADDN P,Q
MOV BL,AL

ADDN R,S
MUL BL

MOV Z,AX

mov ah,4ch
int 21h
end main
```

Output:

Z = 0019H

Explanation: Calculates (2+3) × (4+1) = 25.

---

3. Write an assembly language program for addition of five 16-bit numbers using a PROCEDURE

```
.model small
.stack 100h
.data
A dw 1000h
B dw 2000h
C dw 3000h
D dw 4000h
E dw 5000h
SUM dw ?

.code
main:
mov ax,@data
mov ds,ax

call ADDNUM

mov ah,4ch
int 21h

ADDNUM PROC
MOV AX,A
ADD AX,B
ADD AX,C
ADD AX,D
ADD AX,E
MOV SUM,AX
RET
ADDNUM ENDP

end main
```

Output:

SUM = F000H

Explanation: Adds five 16-bit numbers and stores result in SUM.